

2주차

1. AI의 병렬 연산 필요성

[GPU](#)

[CPU vs GPU](#)

2. GPU 구조(NVIDIA)

3. GPU 실행 계층 구조 및 내부 동작 흐름

[실행 구조](#)

[내부 동작 흐름](#)

4. GPU 메모리 계층 구조

[계층별 메모리 구조](#)

5. GPU 성능 지표

6. 다중 GPU 서버

[PCIe](#)

[NVLink](#)

[NVSwitch](#)

[참고 자료](#)

1. AI의 병렬 연산 필요성

GPU는 대용량 병렬 작업을 처리하기 위해 사용되고, 이는 주로 AI 연산에 사용된다.

GPU

GPU(Graphics Processing Unit)는 화면의 수많은 픽셀을 처리하는 그래픽 API에 종속된 장치였다.

하지만 이러한 대용량 병렬 처리 메커니즘을 그래픽 뿐만 아니라 다른 연산에도 적용하는 방식을 고안하게 되었고,

2006년 NVIDIA가 **CUDA**(Compute Unified Device Architecture)를 공개하면서 **GPU가 그래픽과 무관한 모든 워크로드를 처리하는 것을 가능하게 했다.**

CPU vs GPU

CPU는 **소수의 빠른 클럭 속도와 큰 캐시**를 통해 운영체제의 스케줄링과 같은 **순차적이고 복잡한 작업**을 처리하는 데 최적화 되어있다.

GPU(Graphic Processing Unit)는 수천 개의 작은 코어를 통해 단순 반복 연산을 더 효과적으로 처리할 수 있다.

AI에게 병렬 연산이 필요한 이유는 **행렬 곱셈을 하기 위함**이라고 볼 수 있다.

행렬의 각 원소의 값을 계산하는 과정은 독립적으로 진행된다.

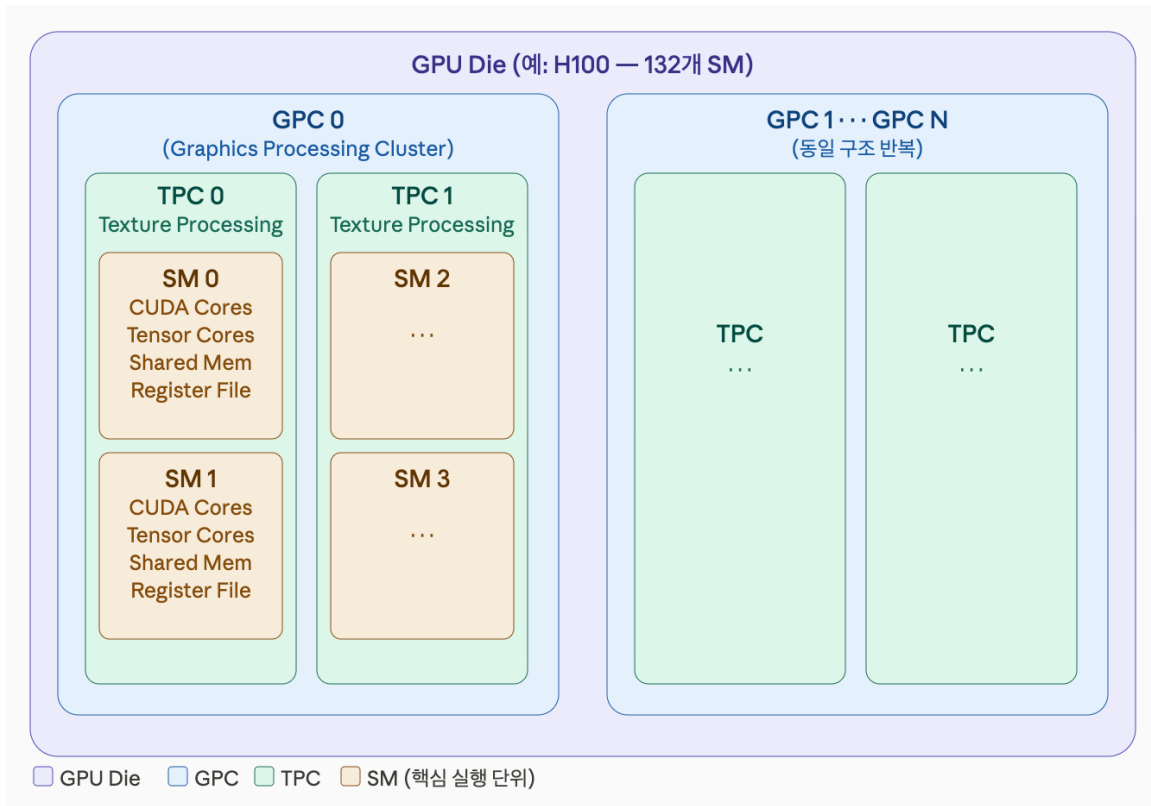
따라서 이는 GPU의 수천 개의 코어를 통해 단순한 곱셈 연산을 동시에 처리하는 것이 효과적이기 때문에 GPU를 사용하는 것이 적합하다.

2. GPU 구조(NVIDIA)

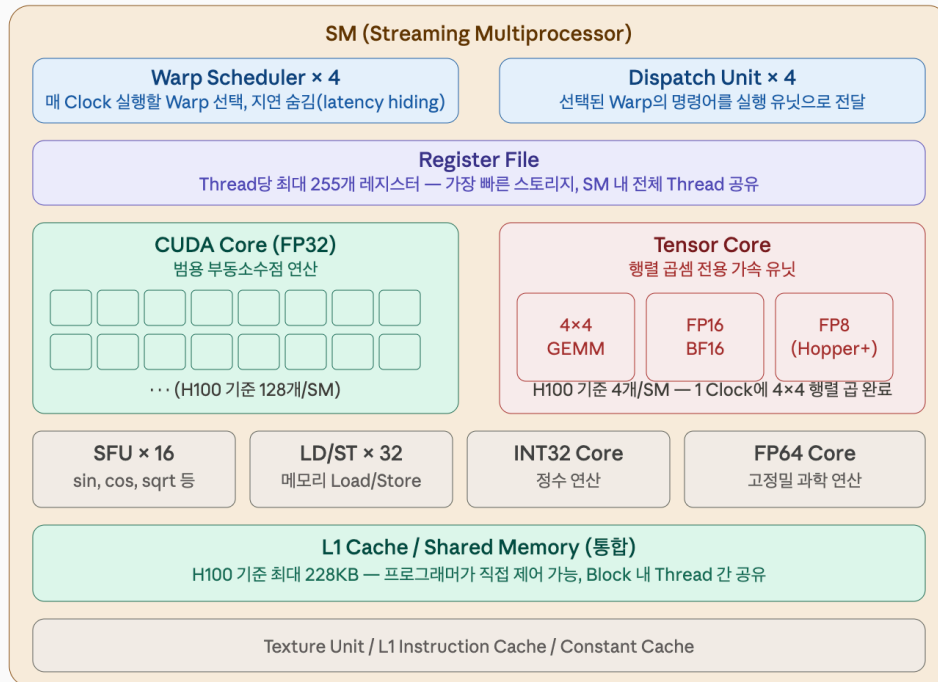
GPU 내부 구조를 정의하는 용어는 다음과 같이 기업마다 다르지만, 계층 구조의 원리는 유사하다.

(NVIDIA 기준으로 학습을 진행했다.)

개념	NVIDIA	AMD	Google TPU
연산 클러스터	GPC	Shader Engine	TensorCore(구글 자체 용어)
핵심 실행 단위	SM	CU (Compute Unit)	—
범용 연산 코어	CUDA Core	Stream Processor	—
행렬 연산 전용	Tensor Core	Matrix Core	MXU (Matrix Multiply Unit)
고속 로컬 메모리	Shared Memory	LDS (Local Data Share)	—
실행 묶음 단위	Warp (32 threads)	Wavefront (64 threads)	—
프로그래밍 모델	CUDA	ROCm / HIP	XLA



GPC(Graphics Processing Cluster)는 여러 개의 **TPC(Texture Processing Cluster)**로 구성되고, TPC는 여러 개의 **SM(Streaming Multiprocessor)**으로 구성된다. GPU에서 실질적인 역할을 하는 것은 이 SM이라고 볼 수 있는데, SM의 내부 구성 요소는 다음과 같다.



• Warp Scheduler



Warp란?

SM(Streaming Multiprocessor)이 실제로 스케줄링 하는 단위를 의미한다. 그리고 GPU에서 동시에 실행되는 쓰레드의 묶음이라고도 볼 수 있다. 기본적으로 32개의 쓰레드를 Warp로 사용한다.
→ 이러한 Warp를 여러 개 묶어둔 것을 쓰레드 블록이라고 한다.

매 Clock마다 Warp를 선택해서 Dispatch Unit으로 전달하는 역할을 한다.

이를 통해 대기 시간을 숨길 수 있는데, 하나의 Warp에 대해 HBM으로 데이터를 요청하게 되면, 데이터를 받기까지 SM에서 지연 시간이 발생한다.

→ 이때, 다른 Warp를 실행함으로써 SM을 효율적으로 작업시키고, 대기 시간을 줄일 수 있다.



이를 나타내는 지표를 Occupancy(점유율)이라고 하는데, 이는 **SM이 수용 가능한 Warp의 수 / 실제 활성 Warp의 수** 비율이다. 즉, **SM이 얼마나 바쁘게 일하는지를 따지는 지표**이다.

- **CUDA Cores**

FP32 부동소수점 연산을 담당하는 범용 코어

- **Tensor Cores**

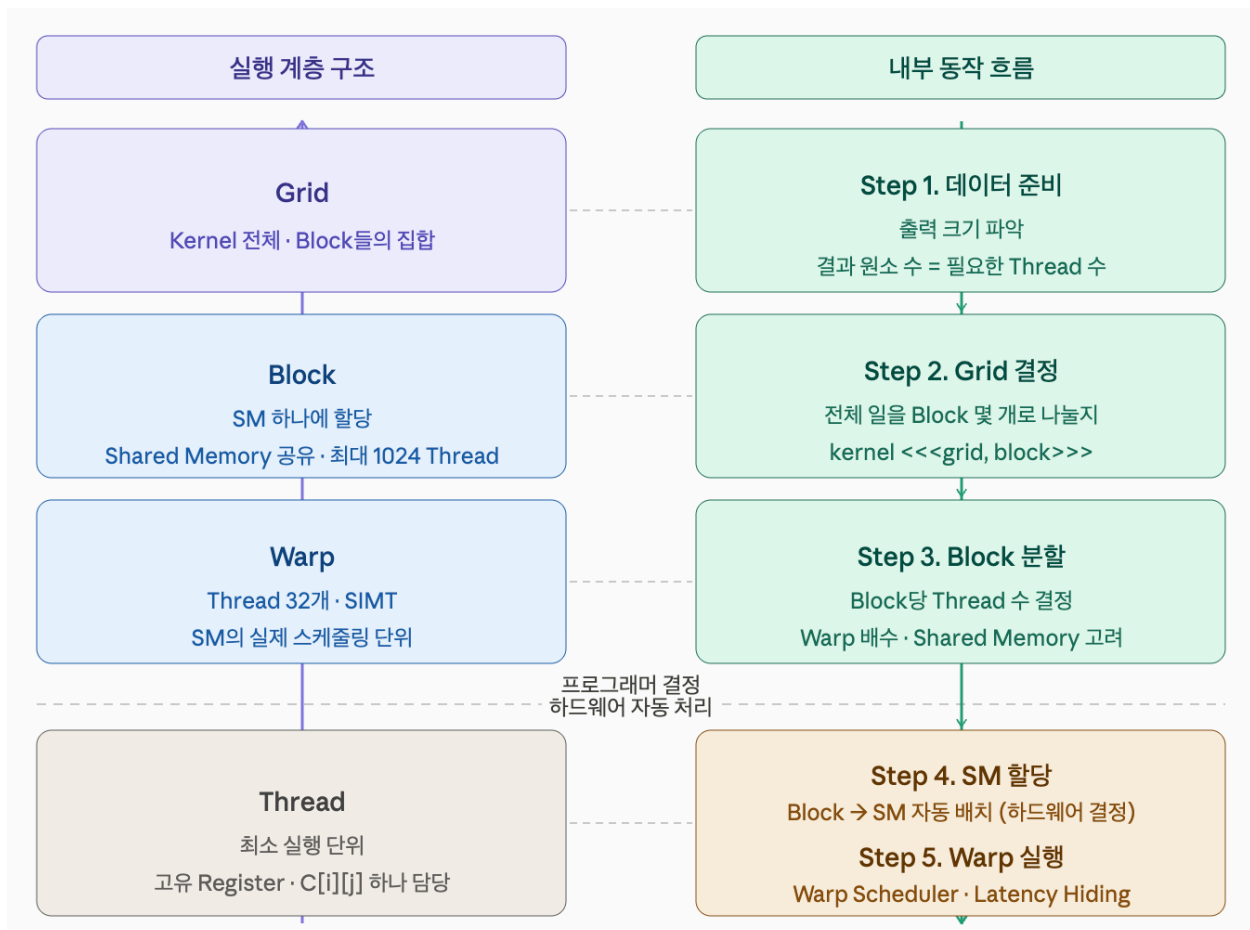
행렬 곱셈 전용 가속 유닛 → 1 Clock : 4×4 행렬 곱셈

- **Shared Memory / L1 Cache**

- **Register File**

- **Dispatch Unit**

3. GPU 실행 계층 구조 및 내부 동작 흐름



실행 구조

1. Thread

GPU에서 최소 실행 단위이다.

2. Warp

32개의 Thread로 구성된 GPU의 기본 실행 단위이다.

3. Block (Thread Block)

한 SM에서 실행되는 스레드의 집합을 의미한다.

여러 스레드 → Warp이므로 스레드 블록은 여러 Warp로 구성되어있다고 볼 수 있다.

동일한 스레드 블록 내의 Warp 간의 공유 메모리를 활용한 메모리를 공유가 가능하다.

4. Thread Block Cluster (Hopper)

Thread Block보다 더 큰 단위로, 스레드 블록 간의 협력으로 이루어지는 실행 그룹이다.

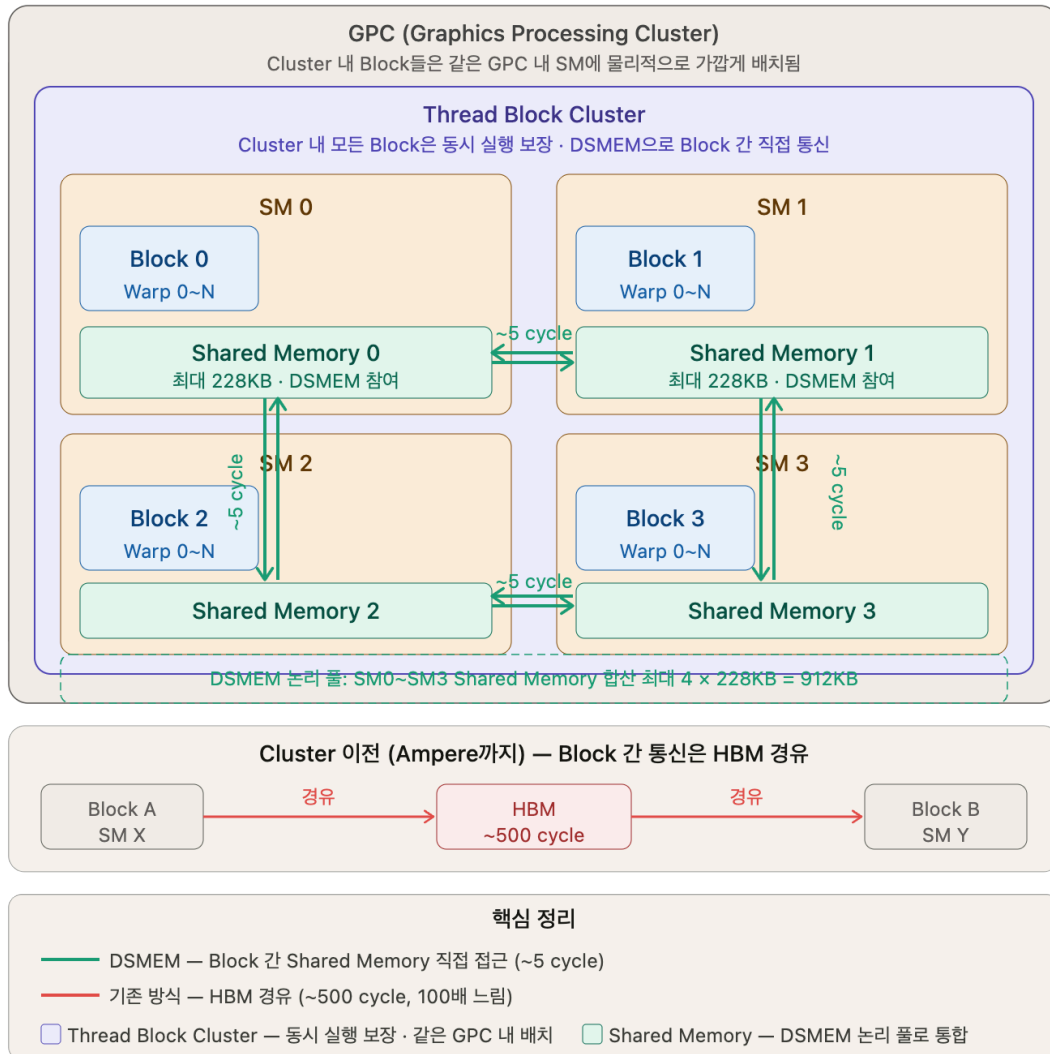


기존 CUDA 실행 계층 (Thread → Warp → Block → Grid) 에서 **Block**은 SM 내부에서 **독립적으로** 실행되었다.

이에 따라 대규모 행렬 연산과 같이 여러 SM이 협력해야 하는 워크로드에서는 **Block 간에 데이터 교환**이 필요했고, 이를 위해서는 전역 메모리(HBM)까지 경유해야 했기 때문에 이 지점에서 병목이 발생하였다.

Thread Block Cluster는 이를 **분산 공유 메모리**(Distributed Shared Memory)라는 개념을 활용하여 해결한다.

Thread Block Cluster + Distributed Shared Memory Hopper (H100) 기준 · Cluster당 최대 16 Block · 같은 GPC 내 배치



이는 여러 하위 실행 단위인 Thread Block들을 여러 SM과 물리적으로 가까이 배치하여 다른 SM 간의 Shared Memory 통신을 가능하게 함으로써 여러 Thread Block들의 협력을 가능하게 한다.

→ 이때, 물리적으로 가까이 배치하는 것은 동일한 GPC 내에 배치하는 것을 의미한다.

5. Grid

여러 블록들의 집합으로, 커널 하나의 전체 실행 범위를 의미한다.

이때, 커널이란 GPU에서 실행하기 위해 호출하는 함수를 의미한다.

내부 동작 흐름

1. 데이터 준비 (CUDA 프로그래머가 결정)

출력 및 문제 크기를 파악한다.

Ex)

- 벡터 덧셈
→ 출력 결과 : 벡터 길이 N ($N = 1024$ 일 경우, Thread 1024개 필요)
- 행렬 곱셈
→ 출력 결과 : 행 x 열 크기 $A * B$ ($A = 1024, B = 1024$ 일 경우, Thread 1048576개 필요)

이와 같이 프로그래머가 결과 수를 결정하는데, 결과 하나를 Thread 하나가 담당하는 구조가 CUDA의 기본 설계 패턴이다.

2. Grid 결정 (CUDA 프로그래머가 결정)

앞서 계산한 결과 수를 Block 단위로 나눈다.

Ex)

- 결과 크기가 원소 1024개, Block 당 Thread의 개수가 128개일 경우, $1024 / 128 = 8$
→ 따라서 Grid의 크기는 8 개가 된다.

3. Block 분할 (CUDA 프로그래머가 결정)

Grid 내부의 Thread Block에서, 하나의 Thread Block 당 몇 개의 쓰레드를 할당할지 결정한다.

주요 고려 요소는 다음과 같다.

- Warp 크기(32)의 배수인가?
배수가 아니라면, 마지막 Warp의 일부 Thread가 유휴 상태가 된다.
(Block의 크기가 100 Thread라면, 마지막에는 고작 4개의 Thread를 위해 Warp 실행)
- Shared Memory 사용량
Block 당 공유 메모리 사용량 증가 시 동시에 수용하는 Block의 수와 점유율 (Occupancy) 감소한다.
- Register Pressure
SM이 수용할 수 있는 Thread 수가 감소하는 경우, Occupancy가 하락한다.

- GPU 활용률

Block이 너무 작을 (ex. 32개의 스레드) 경우, Block 수는 증가하지만 스케줄링 비용도 증가한다.

Block이 너무 클 (ex. 1024개의 스레드) 경우, SM 내의 동시 Block 수가 매우 감소한다.

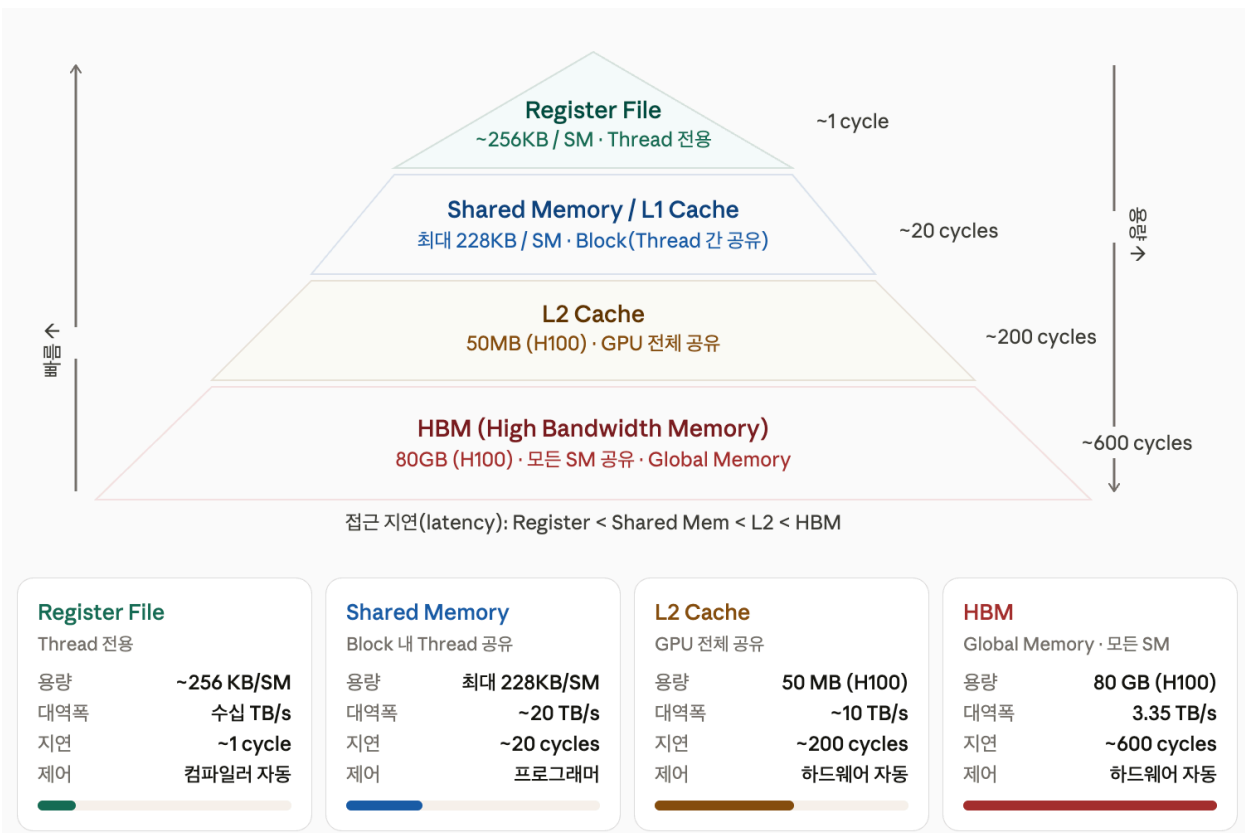
4. SM 할당 (하드웨어가 자동으로 결정)

앞서 설정한 값을 바탕으로 CUDA 커널이 실행되고, 실행 가능한 Block을 SM에 자동으로 배치한다.

5. Warp 실행 (하드웨어가 자동으로 결정)

Block이 SM에 적재되면 내부의 Warp 즉, 32개의 스레드가 실행되고, 이는 Warp Scheduler에 의해 자동으로 관리된다.

4. GPU 메모리 계층 구조



- HBM

GPU 외부에 붙어있는 메인 메모리이다.

- **L2 Cache**

HBM 대신 자주 사용되는 데이터의 캐시 역할을 하는 메모리 계층이다.

- **Shared Memory(Streaming Multiprocessor 내부)**

프로그래머가 직접 제어할 수 있는 계층으로, 행렬 Tilling 시 HBM으로부터 데이터를 불러와 Thread가 재사용하는 방식으로 캐싱할 수 있다.



Tilling이란,

큰 행렬을 작게 나누어(Tile), HBM에서 한 번에 조금씩 Shared Memory에 적재하여 재사용하는 기법이다.

- **Register**

Thread가 계산 중인 값을 임시로 보관하는 공간이다.

계층별 메모리 구조

스코프	메모리 종류	위치	접근 지연
Thread	Register SM당 65,536개 · Thread 전용 지역변수	On-Chip	~1 cycle
Thread	지역 메모리 Register Spill 시 사용 · 컴파일러 자동	Off-Chip (HBM 영역)	~500 cycles
Block / SM	공유 메모리 Block 내 Thread 간 공유 · 직접 제어 가능 Unified Data Cache (최대 256KB, Hopper)	On-Chip	~5 cycles
Block / SM	L1 캐시 공유 메모리와 통합 (Unified Data Cache)	On-Chip	~5 cycles
Grid 전체	전역 메모리 (Global Memory) 모델 가중치 · KV-Cache · 활성화값 H100: 80GB HBM3 · 3.35 TB/s	Off-Chip (HBM3)	~500 cycles
Grid 전체	상수 메모리 (Constant Memory) 읽기 전용 · 64KB 고정 전용 Cache 8KB 보유 (SM 내 On-Chip)	Off-Chip (+전용 Cache)	Hit: ~5 cycles Miss: ~500 cycles
Grid 전체	텍스처 메모리 (Texture Memory) 읽기 전용 · 그래픽스용 · 전용 Cache 보유	Off-Chip (+전용 Cache)	Hit: ~5 cycles Miss: ~500 cycles
GPU 전체 모든 SM 공유	L2 캐시 50MB (H100) · 전역 메모리 버퍼 · 하드웨어 자동	On-Chip	~200 cycles
빠름 · 작음: Register(1) → Shared/L1(5) → L2(200) → Global/Local(500 cycles) :느림 · 큼			

5. GPU 성능 지표

<https://floppeer.io/gpus>

1. FP32

32비트 부동소수점 및 정수의 초당 연산 속도로, CUDA Core의 성능이자 GPU의 **기본 연산 속도**라고 볼 수 있다.

단위는 주로 TFLOPS(Tera Floating Point Operations Per Second)를 사용하는데, 초당 1조 번의 부동소수점 연산을 처리할 수 있다는 의미이다.

2. FP16

16비트 부동소수점 연산 성능으로, **LLM 추론의 표준 정밀도**이다.

Tensor Core를 사용하기 때문에 CUDA Core로 측정되는 FP32보다 처리량이 높다.

3. VRAM(Video RAM)

GPU에 탑재된 HBM의 총 용량을 의미한다.

사용하는 모델에 따라 적절한 GPU를 사용하기 위해서는 VRAM 용량을 계산하는 것이 중요하다.

$$\text{VRAM} = \text{모델 가중치} + \text{KV-Cache} + \text{활성화값(Activation)}$$

4. TDP(Thermal Design Power)

GPU가 최대 부하 시 소비하는 전력을 의미한다.

→ 개인 환경에서는 TDP를 확인하는 것이 중요하다. 데스크탑의 전력 공급 중, GPU의 TDP가 얼마나 차지하는지를 고려해야 한다.

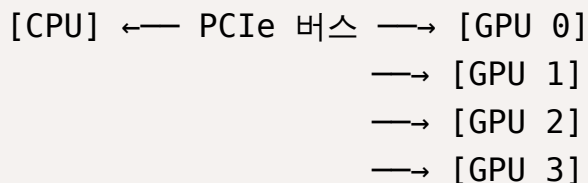
5. Bandwidth (HBM 대역폭)

HBM에서 초당 읽고 쓸 수 있는 데이터량을 의미한다.

6. 다중 GPU 서버

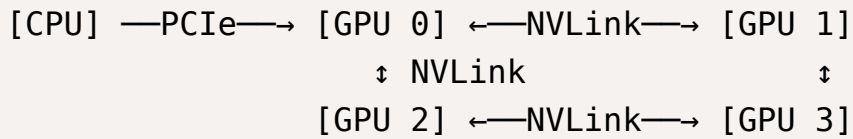
PCIe

내부 하드웨어 구성 요소를 연결하는 표준 인터페이스로, 주변 장치(GPU, SSD, NIC 등)를 연결하는 범용 버스이다.



NVLink

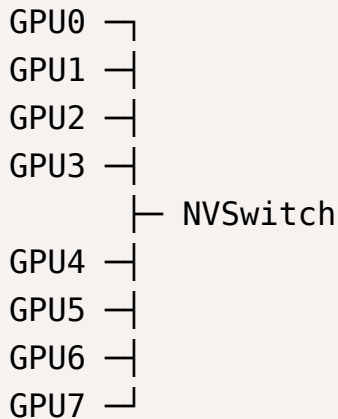
NVIDIA 전용 인터페이스로서, CPU를 경유하지 않고 GPU 간의 직접적인 연결을 제공하는 인터페이스이다.



NVSwitch

NVLink의 상위 개념으로, 여러 GPU를 고속으로 연결하기 위한 스위치다.

기존의 NVLink는 GPU 간의 연결이 1:1로 이루어지지만, NVSwitch는 NVIDIA의 DGX 서버와 같이 여러 GPU를 더욱 대규모 클러스터로 운영할 수 있도록 지원한다.



참고 자료

<https://docs.nvidia.com/cuda/cuda-programming-guide/05-appendices/compute-capabilities.html#compute-capabilities-table-memory-information-per-compute-capability>

https://www.nas.nasa.gov/hecc/support/kb/basics-on-nvidia-gpu-hardware-architecture_704.html?utm_source=chatgpt.com